

BEFORE	AFTER	file
1#include "data.h"	1#include <stdlib.h>	
	2#include "data.h"	
	3	
	4static int imax3(int a, int b, int c)	
	5{	
	6int m = a > b ? a : b;	
	7return m > c ? m : c;	
	8}	
2	9	
3real_t kernel_k53(void)	10real_t kernel_k53(void)	
4{	11{	
	12const long n = LEN_1D;	
	13	
	14/* ---- inspector: index arrays are invariant across nl ---- */	
	15long umax = 0, vmax = 0;	
	16for (long i = 1; i < n; i++) {	
	17if (ju[i] > umax) umax = ju[i];	
	18if (ku[i] > umax) umax = ku[i];	
	19if (jv[i] > vmax) vmax = jv[i];	
	20if (kv[i] > vmax) vmax = kv[i];	
	21}	
	22int *uW = (int *)calloc((size_t)umax + 1, sizeof(int));	
	23int *uR = (int *)calloc((size_t)umax + 1, sizeof(int));	
	24int *vW = (int *)calloc((size_t)vmax + 1, sizeof(int));	
	25int *vR = (int *)calloc((size_t)vmax + 1, sizeof(int));	
	26int *lvl = (int *)malloc((size_t)n * sizeof(int));	
	27int maxL = 0;	
	28for (long i = 1; i < n; i++) {	
	29int a1 = ju[i], b1 = kv[i], a2 = ku[i], b2 = jv[i];	
	30int m = imax3(vW[b1], uW[a1], uR[a1]); /* S1: read v[kv], RMW u[ju]	
	31m = imax3(m, uW[a2], vW[b2]); /* S2: read u[ku], write v[	
	32if (vR[b2] > m) m = vR[b2];	
	33int L = m + 1;	
	34lvl[i] = L;	
	35if (L > maxL) maxL = L;	
	36uW[a1] = L;	
	37if (uR[a2] < L) uR[a2] = L;	
	38if (vR[b1] < L) vR[b1] = L;	
	39vW[b2] = L;	
	40}	
	41free(uW); free(uR); free(vW); free(vR);	
	42	
	43/* stable counting sort of iterations by level */	
	44long *start = (long *)calloc((size_t)maxL + 2, sizeof(long));	
	45long *pos = (long *)malloc(((size_t)maxL + 2) * sizeof(long));	
	46long *order = (long *)malloc((size_t)n * sizeof(long));	
	47for (long i = 1; i < n; i++) start[lvl[i] + 1]++;	
	48for (int L = 1; L <= maxL + 1; L++) start[L] += start[L - 1];	
	49for (int L = 0; L <= maxL + 1; L++) pos[L] = start[L];	
	50for (long i = 1; i < n; i++) order[pos[lvl[i]]++] = i;	
	51free(pos);	
	52free(lvl);	
	53	
	54/* ---- executor ---- */	
5for (int nl = 0; nl < iterations; nl++) {	55for (int nl = 0; nl < iterations; nl++) {	
6for (long i = 1; i < LEN_1D; i++) {	56for (int L = 1; L <= maxL; L++) {	
7u[ju[i]] += v[kv[i]] * c[i];	57long lo = start[L], hi = start[L + 1];	
8v[jv[i]] = u[ku[i]] * d[i] + c[i];	58#pragma omp parallel for schedule(static) if(hi - lo > 512) firstst	
	59for (long k = lo; k < hi; k++) {	
	60long i = order[k];	
	61u[ju[i]] += v[kv[i]] * c[i];	
	62v[jv[i]] = u[ku[i]] * d[i] + c[i];	
	63}	
	64}	
9}	65dummy(a, b, c, d, e);	
10dummy(a, b, c, d, e);	66}	
11}	67	
	68free(start);	
	69free(order);	
12return (real_t)0;	70return (real_t)0;	
13}	71}	